Foundations of High Performance Computing

Lecture 3: ORFEO and HPC software stack

"High performance Computing module

DATA SCIENCE & ARTIFICIAL INTELLIGENCE

SCIENTIFIC & DATA-INTENSIVE COMPUTING

Recap on ORFEO HW

ORFEO today !

IT
Module
IT
Module
(new)
Power
module
Facility for
Epigenomics and
Others

ORFEO HPC nodes before PNRR

TYPE OF NODE	RAM per node	COREs per node	GPUs per node	Peak performance (Tflops)
10 x THIN intel nodes	768 GB	24	-	1,997
2 x FAT intel nodes	1536 GB	36	-	3,46
4 x GPU intel nodes	256 GB	24	**2 x** Nvidia V100 (32GB)	2,073 +2 x 7,0
8 x EPYC Amd nodes	512 GB	128	-	5,03
2 x DGX Nvidia Station	1024 GB	128	8 x Nvidia A100 (40GB)	5,03 + 8 x 9,7 [1]
26 Compute Nodes	**~ 15 TB of RAM**	**1688 cores**	**24 GPUs**	**290 TFlops**

[1] Peak perormance for A100 is reported in FP64, the maximum performance is attained using TF32 (Tensor float) data type, reaching 312 Tera Flops !

ORFEO HPC nodes after last expansion

TYPE OF NODE	RAM per node	CORE s per node	GPU s per node
10 x THIN intel nodes	768 GB	24	-
2 x FAT intel nodes	1536 GB	36	-
4 x GPU intel nodes	256 GB	24	**2 x** Nvidia V100 (32GB)
8 x EPYC Amd nodes	512 GB	128	-
2 x DGX Nvidia Station	1024 GB	128	8 x Nvidia A100 (40GB)
13 x GENOA Amd nodes	512 GB	64	-
3 x Dell XE9680	1024 GB	112	8 x Nvidia H100 (80GB)
42 Compute Nodes	**~ 25 TB of RAM**	2856 **cores**	**48 GPUs**
+ 60 % nodes	**+ 66% more ram**	**+ 70 % more cores**	**+ 100 % new GPUs**

ORFEO network <u>before</u> PNRR

• HPC Network • I/O Network	100 Gbit HDR Infiniband One QM8700 Mellanox: - 16Tb/s aggregate switch throughput - Sub-130ns switch latency
In band Management network	2 x 25Gbit Ethernet 6x dell S5148F switches: - 3.6 Tbps aggregate throughput - Fully redundant network
Out of band Management network	1Gbit Ethernet

ORFEO network after PNRR

• HPC Network • I/O Network	100 Gbit & 200 Gbit Infiniband QM8700 + QM9700 Mellanox - 16 + 51.2 Tb/s aggregate throughput
In band Management network	2 x 25Gbit Ethernet link 10x dell S5148F switches: - 3.6 Tbps aggregate throughput - Fully redundant network
Out of band Management network	1Gbit Ethernet

ORFEO storage before PNRR

	Standard storage (HDD)	Fast storage (SSD)	Super Fast storage (NVME)	Long Term Storage
Node capacity	(Gen1) **212 TB** (Gen2) **294 TB**	**84 TB**	**2 x 1.6 TB**	**2962.5 TB**
# of Nodes	**6 x** Gen1 + **2 x** Gen2 nodes	**4** nodes	**2** devices each node	Served via NFS server with a SAN.
Storage provider	CEPH	CEPH	CEPH	Network FS (NFS)
RAW storage	**~ 2450 TB**	**~ 340 TB**	**~ 32 TB**	**~ 2962 TB**

ORFEO storage after PNRR

	Standard storage (HDD)	Fast storage (SSD)	Super Fast storage (NVME)	Long Term Storage
Node capacity	(Gen1) **212 TB** (Gen2) **294 TB** (Gen3) **294 TB**	**84 TB**	**2 x 1.6 TB** **2 x 14 TB** (Gen3 nodes)	**2962.5 TB**
# of Nodes	**6 x** Gen1 + **2 x** Gen2 + **11 x** Gen3	**4** nodes	2 devices each node, **Gen3 nodes have 2x 14 TB !**	Served via NFS server with a SAN.
Storage provider	CEPH	CEPH	CEPH	Network FS (NFS)
	+ 130 % more HDD storage		**+ 1340 % more NVME storage**	
RAW storage	**~ 5684 TB**	**~ 340 TB**	**~ 340 TB**	**~ 2962 TB**

I/O system on ORFEO

The storage is provided mainly by:

- A **CEPH** cluster consisting of 23 nodes equipped with HDD, SSD, and NVMe technologies
- A Dell PowerVault **SAN**, connected via Fibre Channel to an NFS server.
- A Dell ML3 **Tape Library** to archive data on LTO-9 cartridges, each offering up to 45 TB of capacity.

Ceph filesystem on ORFEO:

/orfeo/cephfs/home

Host the user's home.
A quota of 200GB and 10^6 files is enforced.
It run on **SSD with replica 3 data protection.**

/orfeo/cephfs/scratch

It is large area intended to be used to store data that need to be elaborated.
It is also physically located on ceph large FS, and exported via infiniband to all the computational nodes.
It run on **HDD** and **6+2 EC data protection**

/orfeo/cephfs/fast

is a fast space available for each user, on all the computing nodes.
is intended to be a **fast scratch area** for data intensive application.
It run on **SSD with replica 3 data protection.**

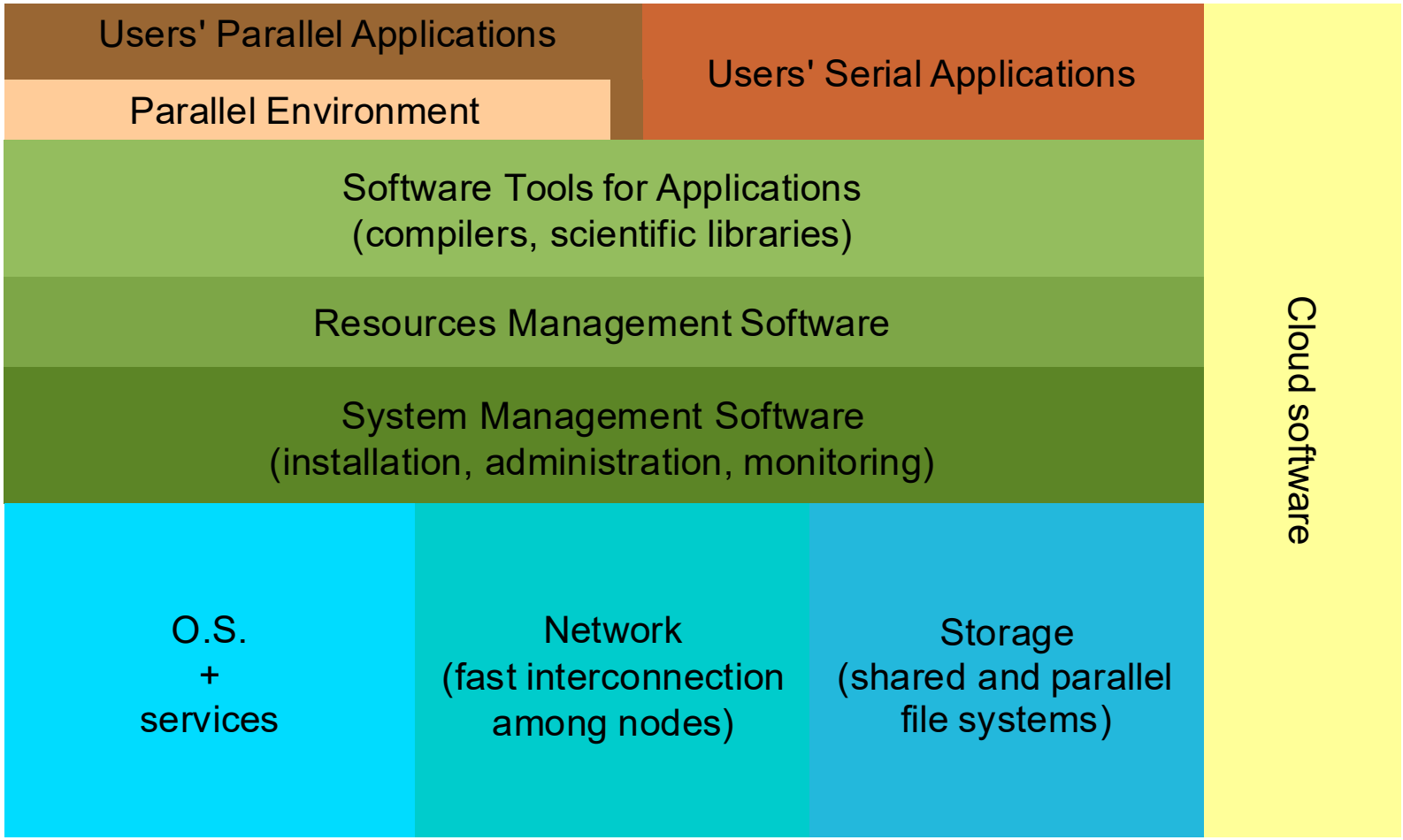
Interact with ORFEO storage..

```
> pwd
> /usr/local/bin/myquota
> df | grep -v user | grep -v tmp
> cd fast/
>  dd if=/dev/zero of=testfile bs=2M count=1024 conv=fdatasync
> dd if=/dev/zero of=testfile bs=4M count=1024 conv=fdatasync
> cd scratch/
> dd if=/dev/zero of=testfile bs=2M count=1024 conv=fdatasync
 > dd if=/dev/zero of=testfile bs=4M count=1024 conv=fdatasync
```

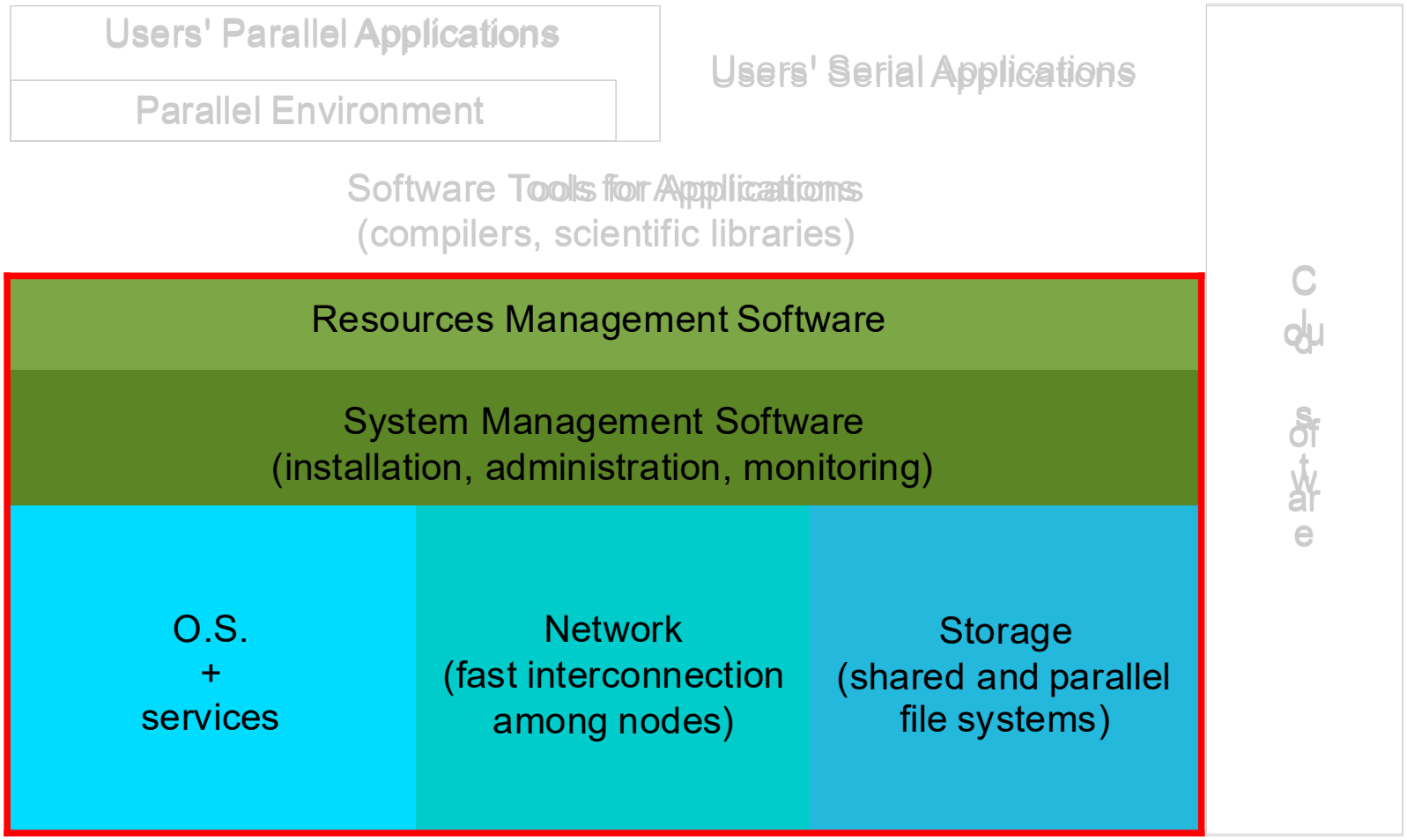
Agenda

- A first look of the software stack
- Local resource manager: queue system
- Scientific software
- Compilers
- Libraries

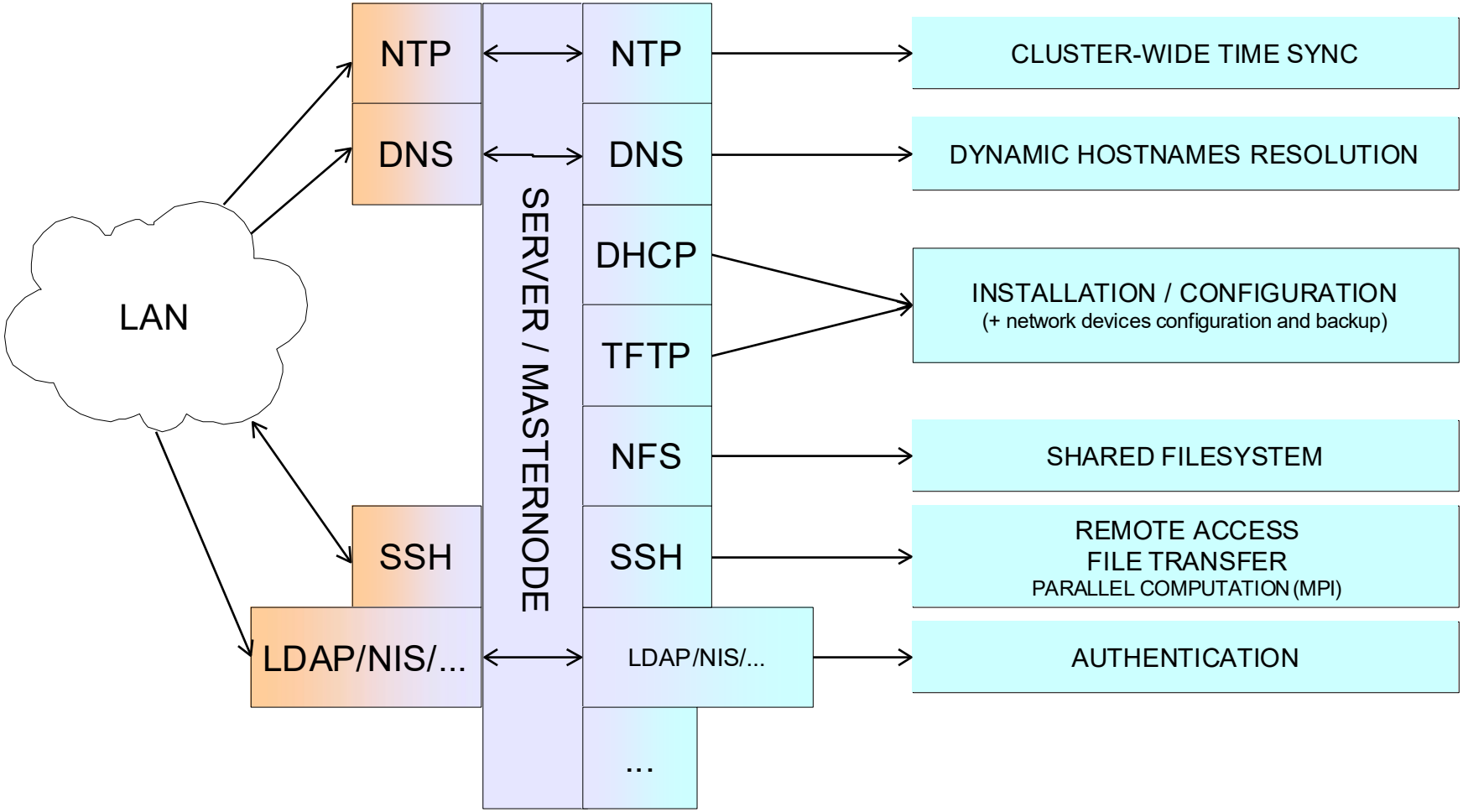
What we need..


Users' Parallel Applications
Parallel Environment
Users' Serial Applications
Software Tools for Applications
(compilers, scientific libraries)
Resources Management Software
System Management Software
(installation, administration, monitoring)
O.S.
+
services
Network
(fast interconnection
among nodes)
Storage
(shared and parallel
file systems)
Cloud software

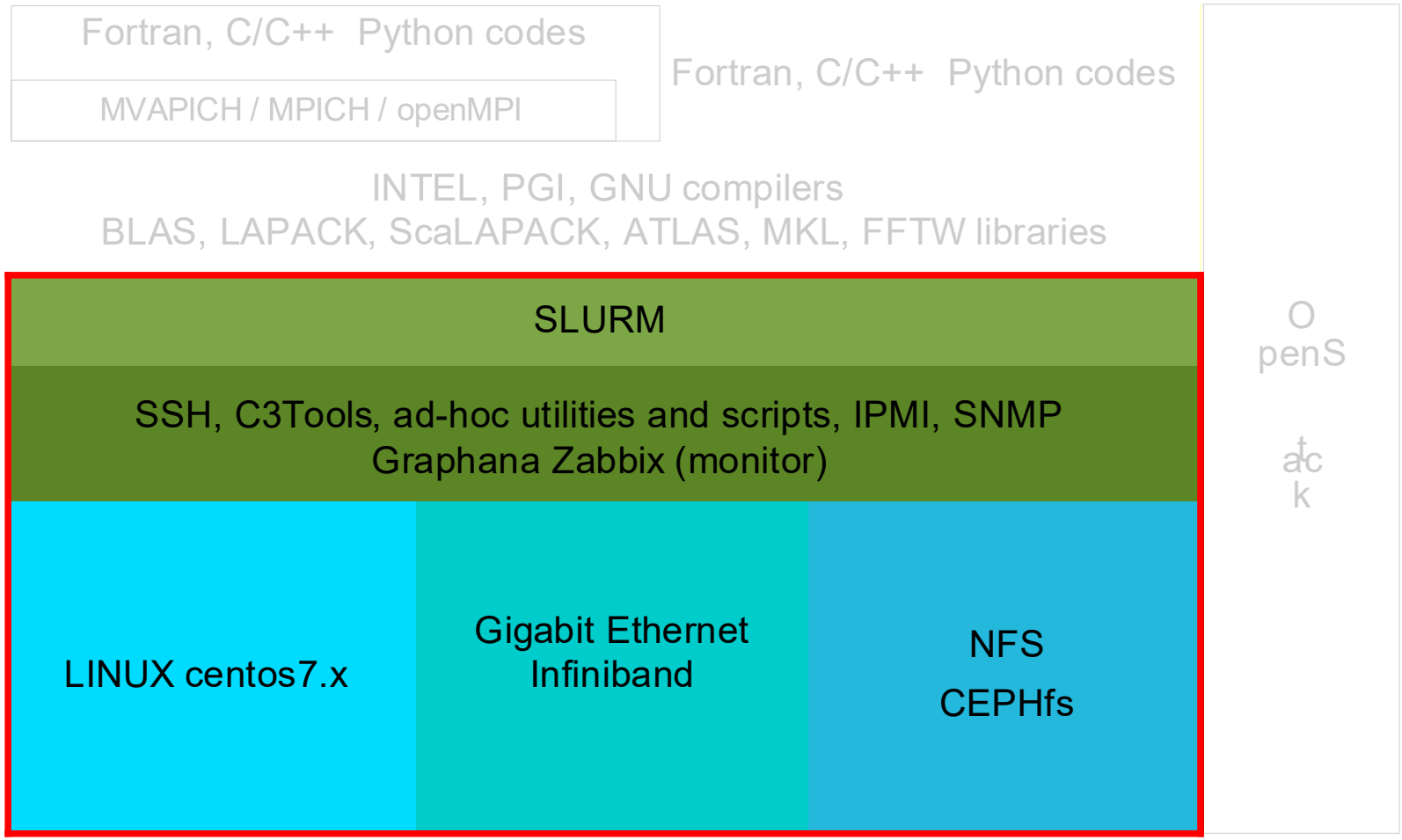
The cluster middleware



Cluster wide services

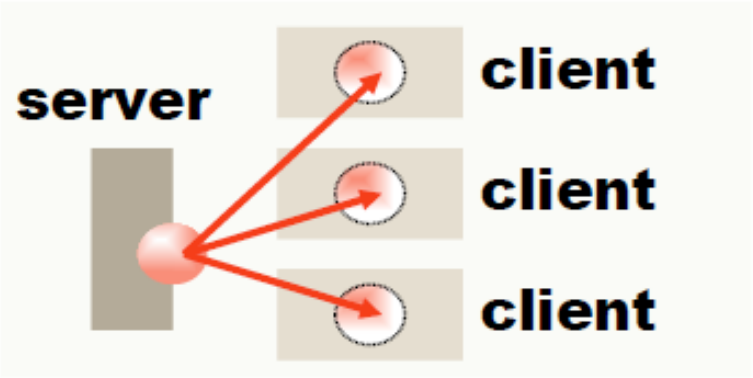


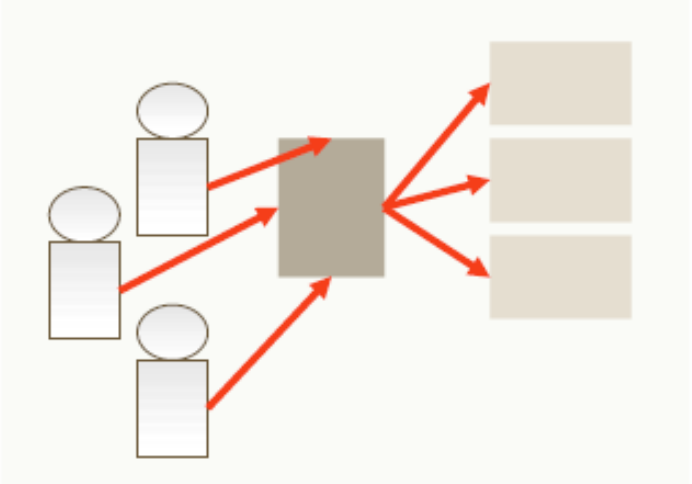
Middleware software used on ORFEO



Cluster middleware

- Administration software:
  - user accounts
  - NTP/NFS/ etc…
- Resource management and scheduling software (LRMS)
  - Process distribution
  - Load balance
  - Job scheduling of multiple tasks





Agenda

- A first look of the software stack ✓
- Local resource manager: queue system
- Scientific software
- Compilers
- Libraries

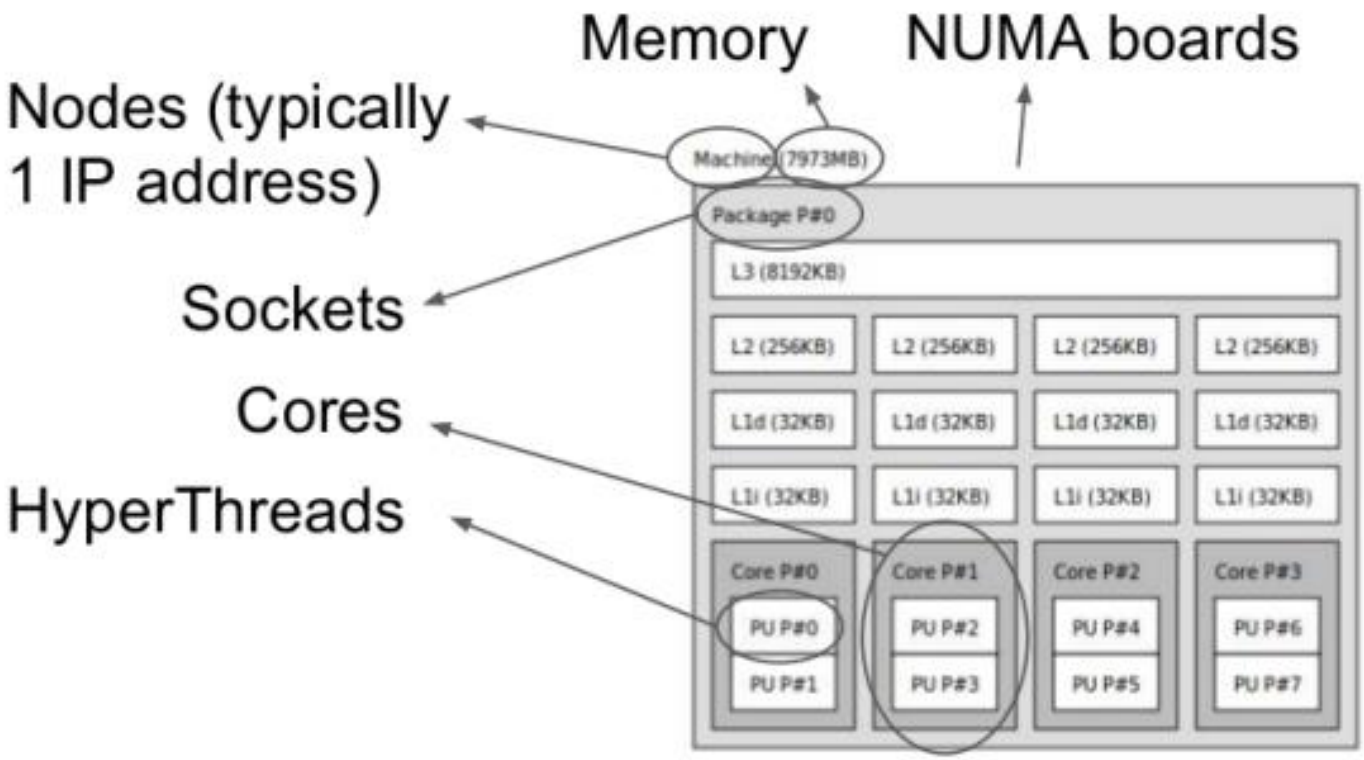
Resource Management Problem

- We have a pool of users and a pool of resources, then what?
  - some software that controls available resources
  - some other software that decides which application to execute based on available resources
  - some other software devoted to actually execute applications

What are we speaking about ?

REPLACE THE CAKE WITH HPC RESOURCE

HPC resources..



PLUS:

- network resources
- GPU/Accelerator
- Software resources

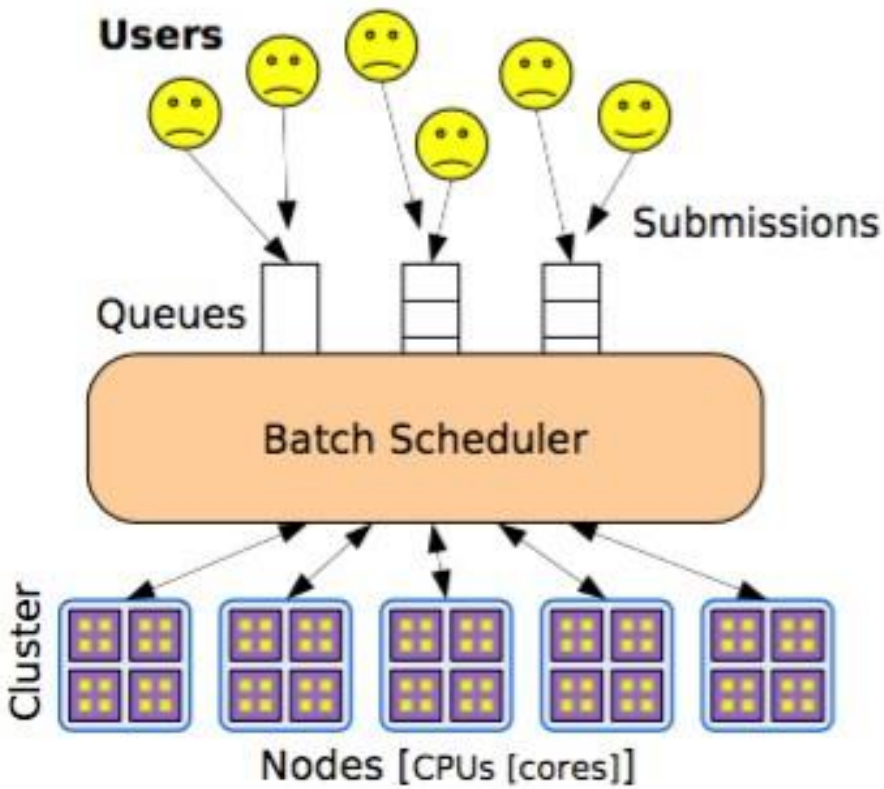
Some definition

- **Batch Schedule**r: software responsible for scheduling the users' jobs on the cluster.

  scheduling is the method by which work specified by some means is assigned to resources that complete the work

- **Resources Manage**r: software that enable the jobs to connect the nodes and run.
- **Node** (aka **Computing Node**): computer used for its computational power.
- **Login/Master node**: it's through this node that the users will submit/launch/manage jobs.

Batch scheduler



- Allocate resources for each applications with respect of their requirements and users' rights.

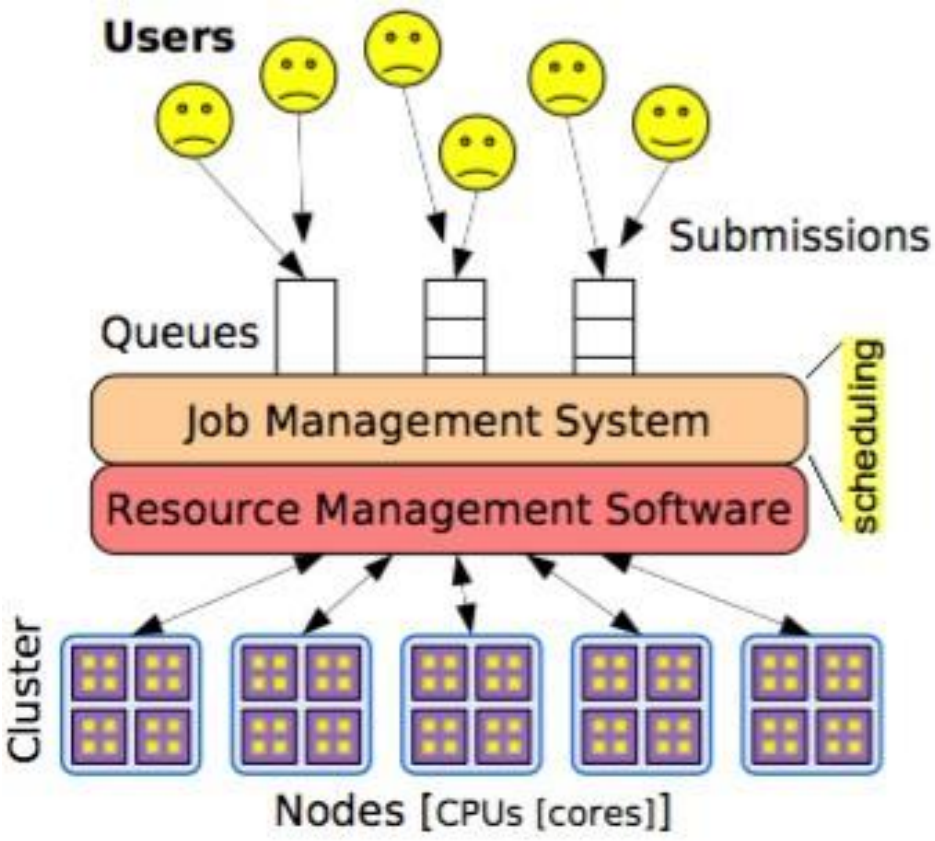
→ Satisfy users:

response time,reliability

→ Satisfy admins

high resource utilization efficiency,energy management

Batch scheduler (2)

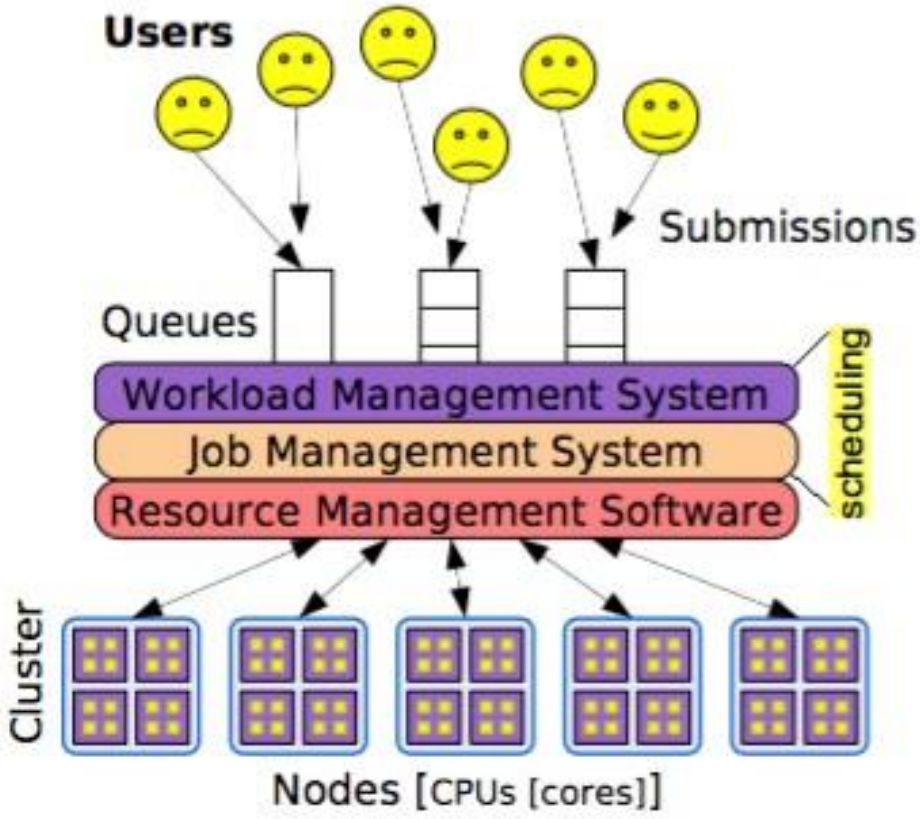


Resource Management Layer
→ launching, cleaning, monitoring

Job Management Layer

→batch/interactive job
→backfilling
→scheduling
→suspend/Resume
→preemption
→dependencies
→resubmission
→advance reservation

Batch scheduler (3)



Workload/Job Management

→ more complete job scheduling policies

→ Fairsharing, Quality of Service (QoS), SLA (Service Level Agreement), Energy Saving

→ Sometime a dedicated software

Main LRMS packages

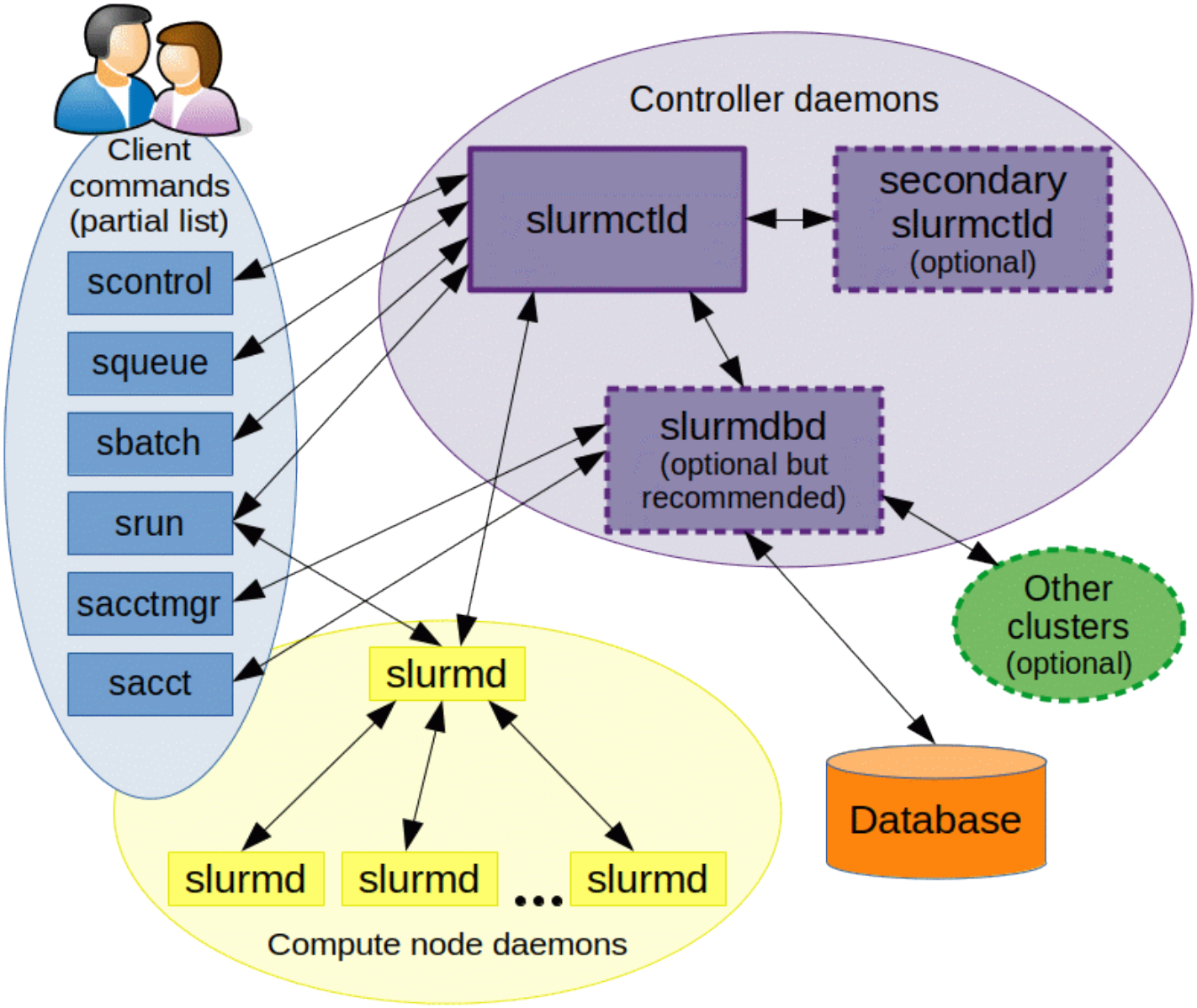
- IBM LSF
  - commercial
- Univa Grid Engine (UGE)
  - Commercial originates from SGE
- PBSPRO
  - Portable Batch System Professional once commercial now open
  - Support is commercial
  - **no longer Available on ORFEO**
- SLURM
  - Open source
  - Support is commercial
  - **Available on ORFEO just for you**

What is SLURM ?

- Historically, Slurm was an acronym of:
- – Simple Linux Utility for Resource Management
- Dev. started in 2002 @ Lawrence Livermore National Lab as a resource manager for Linux clusters
- Sophisticated scheduling plugins added in 2008
- About 550,000 lines of C code today
- Used on many of the world's largest computers
- Active global user community

SLURM slides taken and slightly adapted from: slurm_overview.pdf (schedmd.com)

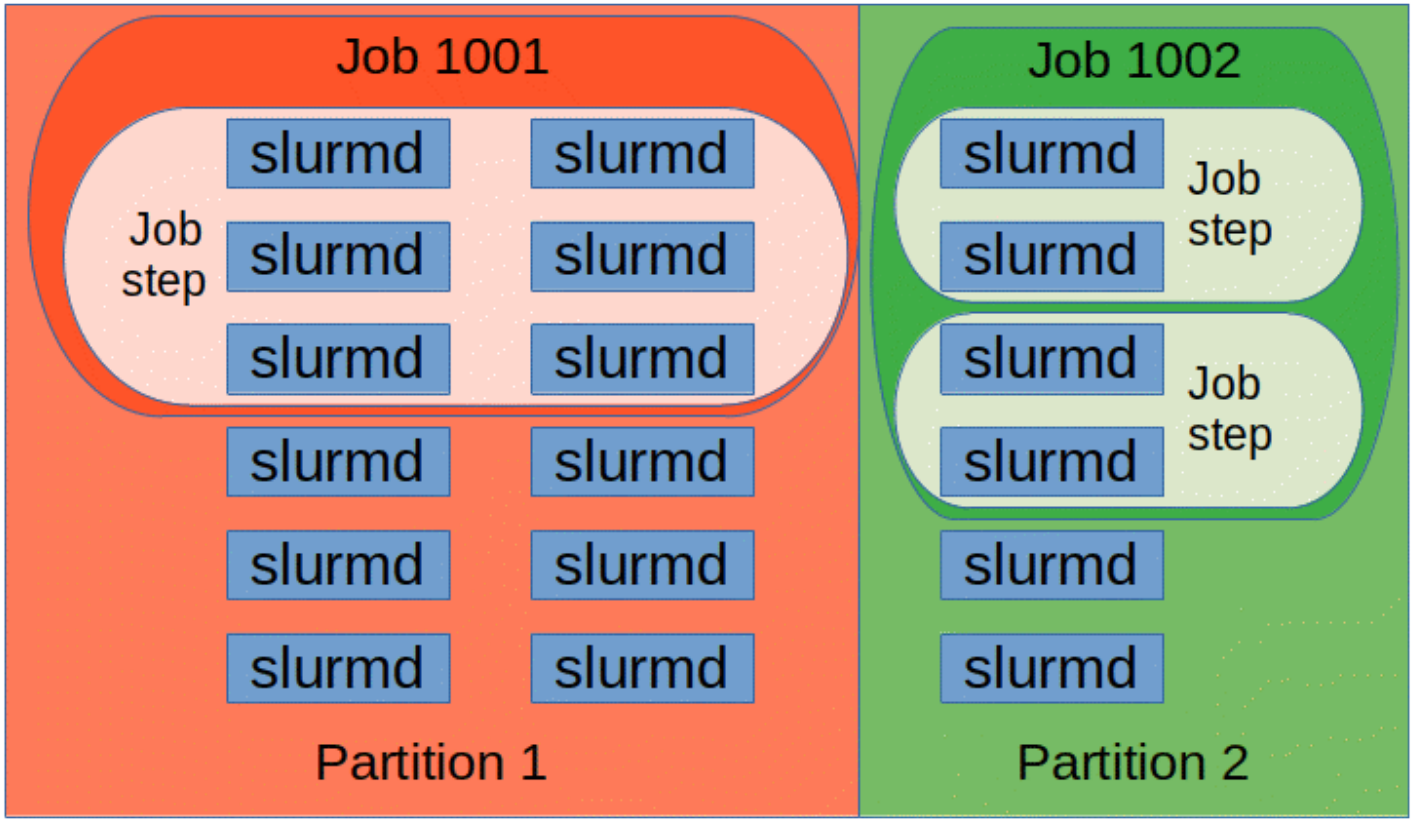
Slurm architecture



SLURM entities

- **nodes**, the compute resource in Slurm,
- **partitions**, which group nodes into logical (possibly overlapping) sets,
- **jobs**, or allocations of resources assigned to a user for a specified amount of time
- **Job steps**, which are sets of (possibly parallel) tasks within a job.
- **Qos:** Limits and policies

Slurm entities



A slurm jobfile

SLURM directives using the tag **#SLURM**, describe the job requirments in terms of execution queue, number of nodes and cores, job name, walltime, etc.	#!bin/bash #SLURM --ntasks=128 #SLURM --cpus-per-task=2 #SLURM --mem-per-cpu=20 #SLURM --time=60
The rest of the job is a standard shell script SLURM "lands" user's home directory: it is important to change the directory to the one in which we want to run the job	cd $HOME/MyJobDir hostname pwd

Slurm Cheatsheet

https://slurm.schedmd.com/pdfs/summary.pdf

slurm
cheat sheet

Daemons

slurmctld	Executes on cluster's"head" node to manage workload.
slurmd	Executes on each compute node to locally manage resources.
slurmdbd	Manages database of resources limits, licenses, and archives accounting records.
slurmrestd	Interface to Slurm via REST API.

Job Submission

sbatch - Submit a batch script for later execution.

srun - Obtain a job allocation (as needed) and execute an application.

salloc - Obtain a job allocation.

--array=<indexes> (e.g. "--array=1-10")	Job array specification.
--account=<name>	Account to be charged for resources used.
--begin=<time>(e.g. "--begin=18:00:00")	Initiate job after specified time.
--clusters=<name>	Cluster(s) to run the job. (sbatch command only)
--constraint=<features>	Required node features.
--cpus-per-task=<count>	Number of CPUs required per task.
--dependency=<state:jobid>	Defer job until specified job(s) reach specified state.
--error=<filename>	File in which to store job error messages.
--exclusive[=user]	Allocated nodes can not be shared with other jobs/users.
--export=<name[=value]>	Export identified environment variables.
--gres=<name[:count]>	Generic resources required per node.
--job-name=<name>	Job name.
--label	Prepend task ID to output. (srun command only)
--licenses=<name[:count]	License resources required for entire job.
--mem=<MB>	Memory required per node.
--mem-per-cpu=<MB>	Memory required per allocated CPU.
-N<minnodes[-maxnodes]>	Node count required for the job.
-n<count>	Number of tasks to be launched.
--nodelist=<names>	Specific host names to include in job allocation.
--output=<name>	File in which to store job output.
--partition=<names>	Partition/queue in which to run the job.
--qos=<name>	Quality Of Service.
--signal=[B:]<num>[@time]	Signal job when approaching time limit.
--time=<time>	Wall clock time limit.
--wrap=<command_string>	Wrap specified command in a simple "sh" shell. (sbatch command only)

Accounting

sacct - Display accounting data.

--allusers	Displays all users' jobs.
--accounts=<name>	Displays jobs with specified accounts.
--endtime=<time>	End of reporting period.
--format=<spec>	Specify the fields shown in the output.
--partition=<names>	Comma separated list of partitions to select jobs and job steps from.
--state=<state_list>	Display jobs with specified states.
--starttime=<time>	Start of reporting period.
--long	Provides output with more fields listed. .

sacctmgr - View and modify account information.

Options:

--immediate	Commit changes immediately.
--parsable	Output delimited by '|'

Commands:

add <ENTITY> <SPECS> create <ENTITY> <SPECS>	Add an entity. Identical to the create command.
delete <ENTITY> where <SPECS>	Delete the specified entities.
show <ENTITY> [<SPECS>]	Display information about the specific entity.
modify <ENTITY> where <SPECS> set <SPECS>	Modify an entity.

Entities:

account	Account used to group users.
association	Entity on cluster, consisting of cluster, account, user and (optionally) partition.
cluster	ClusterName parameter in the slurm.conf.
qos	Quality of Service.

sreport - Report job usage and cluster utilization.

--allusers	Displays all users' jobs.
--accounts=<name>	Displays jobs with specified accounts.

Report Types:

cluster	AccountUtilizationByUser, UserUtilizationByAccount, UserUtilizationByWckey, Utilization, WCKeyUtilizationByUser
job	SizesByAccount, SizesByAccountAndWckey, SizesByWckey
reservation	Utilization
user	TopUsage

SCHEDMD
The Slurm Company

Copyright 2023 SchedMD LLC. All rights reserved.
www.schedmd.com

Job Management

squeue - View information about jobs.

--account=<name>	View only jobs with specified accounts.
--clusters=<name>	View jobs on specified clusters.
--format=<spec> (e.g. *--format =%i,%j*)	Output format to display. Specify fields, size, order, etc.
--jobs<job_id_list>	Comma separated list of job IDs to display.
--name=<name>	View only jobs with specified names.
--partition=<names>	View only jobs in specified partitions.
--priority	Sort jobs by priority.
--qos=<name>	View only jobs with specified Qualities Of Service.
--start	Report the expected start time and resources to be allocated for pending jobs in order of increasing start time.
--state=<names>	View only jobs with specified states.
--users=<names>	View only jobs for specified users.

sinfo - View all information about nodes and partitions.

--all	Display information about all partitions.
--dead	If set, only report state information for non-responding (dead) nodes.
--format=<spec>	Output format to display.
--iterate=<seconds>	Print the state at specified interval.
--long	Print more detailed information.
--Node	Print information in a node-oriented format.
--partition=<names>	View only specified partitions.
--reservation	Display information about advanced reservations.
-R	Display reasons nodes are in the down, drained, fail or failing state.
--state=<names>	View only nodes in specified states.

scancel - Signal jobs, job arrays, and/or job steps.

--account=<name>	Operate only on jobs running in the specified account.
--name=<name>	Operate only on jobs with specified name.
--partition= <names>	Operate only on jobs in the specified partition/queue.
--qos=<name>	Operate only on jobs using the specified quality of service.
--reservation= <name>	Operate only on jobs using the specified reservation.
--state=<names>	Operate only on jobs in the specified state.
--user=<name>	Operate only on jobs from the specified user.
--nodelist= <names>	Operate only on jobs using the specified compute nodes.

scontrol - Used view and modify configuration and state. Also see the **sview** graphical user interface version.

--all	When show command is used, then display all partitions, their jobs, and job steps.
--details	Causes show command to provide more details when available.
--future	Report nodes in FUTURE state.
--oneliner	Print information on one line.

Commands:

create *SPECIFICATION*	Create a new partition or reservation.
delete *SPECIFICATION*	Delete the entry with the specified SPECIFICATION.
reconfigure	All Slurm daemons will re-read the configuration file.
requeue JOB_LIST	Requeue a running, suspended or completed batch job.
setdebugflags	Add or remove DebugFlags of the slurmctld daemon.
show ENTITY ID	Display the state of the specified entity with the specified identification.
update *SPECIFICATION*	Update job, step, node, partition, or reservation configuration per the supplied specification.

Environment Variables

SLURM_ARRAY_JOB_ID	Set to the job ID if part of a job array.
SLURM_ARRAY_TASK_ID	Set to the task ID if part of a job array.
SLURM_CLUSTER_NAME	Name of the cluster executing the job.
SLURM_CPUS_PER_TASK	Number of CPUs requested per task.
SLURM_DEBUG_FLAGS	Specify debug flags for sbatch to use.
SLURM_JOB_ACCOUNT	Account name.
SLURM_JOB_ID	Job ID.
SLURM_JOB_NAME	Job Name.
SLURM_JOB_NODELIST	Names of nodes allocated to job.
SLURM_JOB_NUM_NODE	Number of nodes allocated to job.
SLURM_JOB_PARTITION	Partition/queue running the job.
SLURM_JOB_UID	User ID of the job's owner.
SLURM_JOB_USER	User name of the job's owner.
SLURM_RESTART_COUNT	Number of times job has restarted.
SLURM_PROCID	Task ID (MPI rank).
SLURM_STEP_ID	Job step ID.
SLURM_STEP_NUM_TASKS	Task count (number of MPI ranks).

SCHEDMD
The Slurm Company

Copyright 2023 SchedMD LLC. All rights reserved.
www.schedmd.com

https://hpc-tutorials-c907cb.gitlab.io/Introduction/using_slurm/

Recap on LRMS

- LRMS is a fundamental tool in the HPC management:
  - User: know it well and you will almost run !
  - Sys. Adm.: know it well and you will keep your system busy..
- Many different choices
- Concepts are similar /commands sometime also (to help survive: http://www.schedmd.com/slurmdocs/rosetta.pdf, available on our repo)
- Key point is THE scheduler
  - Theoretically is almost all possible in resource scheduling with modern LRMS software to accommodate requests from users
  - Practically is almost impossible satisfy all your users (and/or communities)

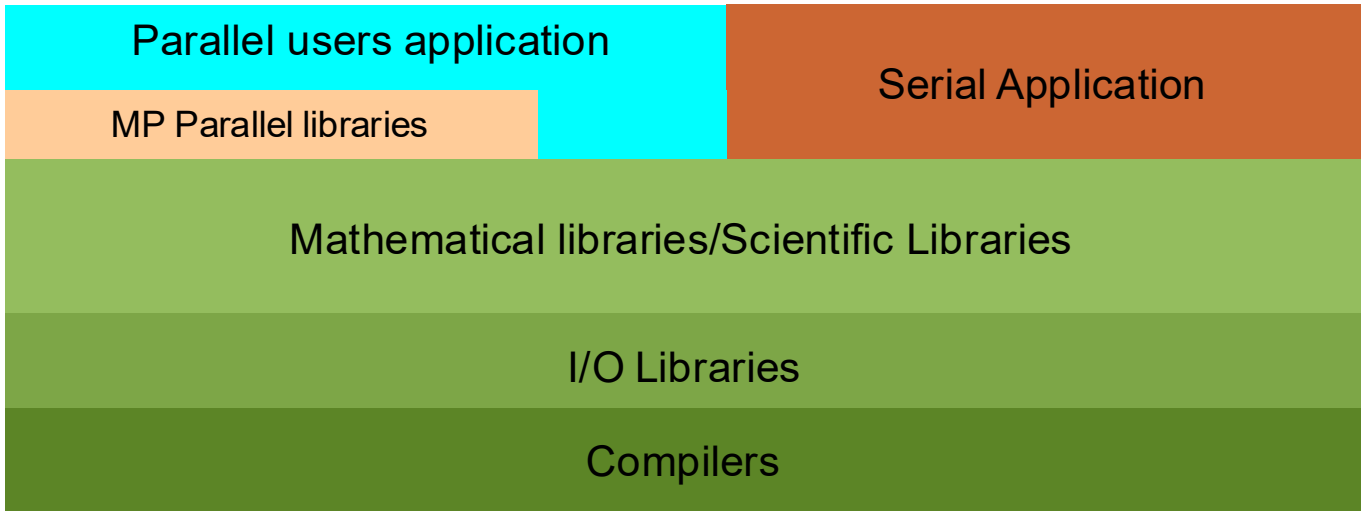
Resource sharing policies is not at all a technical problem !

Agenda

- A first look of the software stack ✓
- Local resource manager: queue system ✓
- Scientific software
- Compilers
- Libraries

HPC scientific Software layers (interleaved..)

- User's applications (both parallel and serial)
- Parallel Libraries&Tools
- Mathematical/Scientific Libraries
- I/O libraries
- Compilers



HPC software

- Not much standardization in HPC: every machine/app has a different software stack
  - This is done to get the best performance
- HPC frequently trades reuse and usability for performance
  - Reusing a piece of software frequently requires you to port it to many new platforms
- List of packages/combination can diverge...

Dependency Nightmare..

Scientific software: where is ?

- Generally available cluster-wide
- installed in /opt/cluster/software (or similar) and mounted read-only on the nodes via nfs
- Generally managed by modules package
- Several versions managed by some agreement

Module package (1)

- Modules allow to dynamically modify user environment
- Useful tool to track different version of installed software

Module package (2)

- A few useful commands

`module avail` – lists all available modules

`module list` – lists all loaded modules

`module load` – adds a module to your environment

`module unload` – removes a module from your environment

Module and environment

- Module command change on the fly the most important ENVIROMENT VARIABLE for you
  - PATH
  - LD_LIBRARY_PATH

ORFEO situation

```
[cozzini@login01 scratch] > module avail

------------------------------------------------- /orfeo/opt/modules/tools -------------------------------------------------
   IGV/2.18.0                fastp/0.23.4                java/21.0.2         (D)    plink/1.90
   R/4.4.1                   fastp/0.24.1        (D)     jupyterlab/4.4.7           sambamba/1.0
   R/4.5.0           (D)     fastqc/0.12.1               ont-guppy-cpu/6.2.1        sambamba/1.0.1     (D)
   STAR/2.7.11b              foldseek/8-ef4e960          ont-guppy-cpu/6.5.7 (D)    samtools/1.17
   bcftools/1.17             foldseek/10-941cd33 (D)     ont-guppy-gpu/6.2.1        samtools/1.21      (D)
   bcftools/1.21     (D)     gromacs/2025.1              ont-guppy-gpu/6.5.7 (D)    singularity/3.10.4
   bcl2fastq2/2.20.0         hwloc/2.12.0                openBLAS/0.3.26-omp        singularity/3.11.5
   bedtools2/2.31.1          java/1.8.0                  openBLAS/0.3.26            singularity/4.3.1  (D)
   bwa-mem2/2.2.1            java/8-8u402b06             openBLAS/0.3.29-omp        trim_galore/0.6.10
   cutadapt/4.2              java/11.0.22                openBLAS/0.3.29     (D)    vscode/4.106.2
   cutadapt/5.0      (D)     java/17.0.10                picard/3.4.0

------------------------------------------------- /orfeo/opt/modules/mpi ---------------------------------------------------
   openMPI/4.1.6    openMPI/5.0.5 (D)

------------------------------------------------ /orfeo/opt/modules/libraries ----------------------------------------------
   cuda/11.8    cuda/12.1    cuda/12.8        (D)    openBLAS/0.3.26-omp    openBLAS/0.3.29-omp
   cuda/12.0    cuda/12.6    cutensor/2.2.0.0        openBLAS/0.3.26        openBLAS/0.3.29

  Where:
   D:  Default Module
```

Agenda

- A first look of the software stack ✓
- Local resource manager: queue system ✓
- Scientific software ✓
- Compilers
- Libraries

What does mean compiling ?

- A complex translation from high level language (C/Fortran…) to a stream of instructions..

Compilers

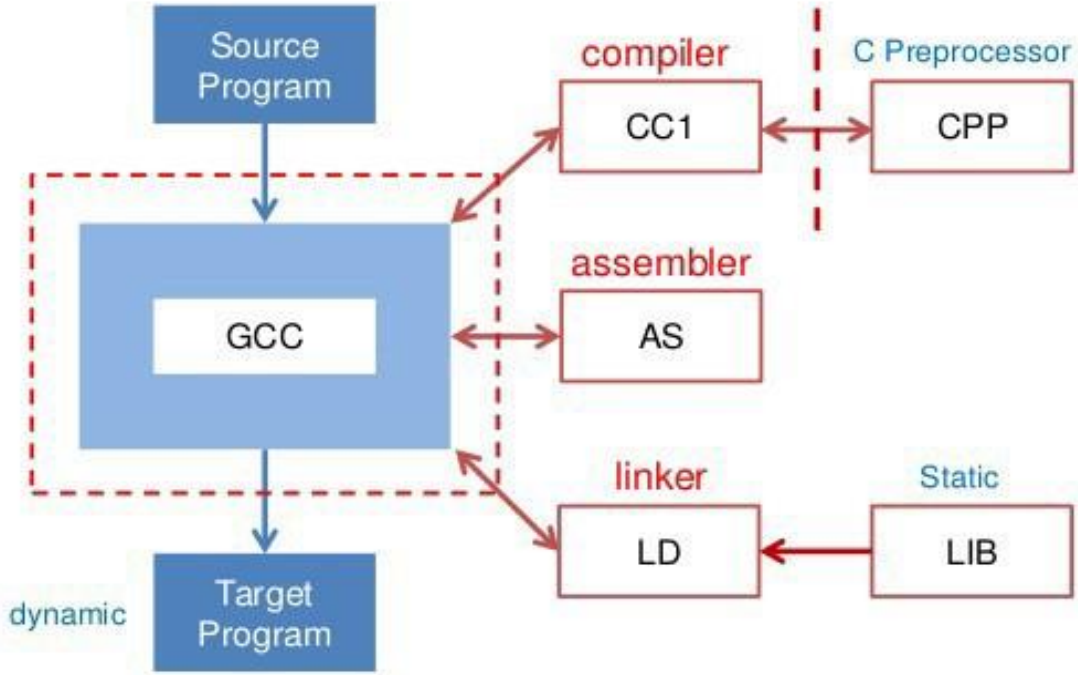
- Free : Gnu suite
  - Always available
  - Many different versions
  - Fundamental but some time lacks performance
- Commercial compilers for free: (you pay support)
  - Intel suite :
    - A full software stack (includes libraries/ profiling /benchmarking tools /MPI libraries)
    - highly optimized
  - Nvidia HPC SDK (previously know as PGI)
    - A full software stack (includes libraries/ profiling /benchmarking tools for CPU and GPUs)
    - Available also for ARM architecture

What is available on ORFEO ?

```
[cozzini@login01 scratch] > gcc -v
Using built-in specs.
COLLECT_GCC=gcc
COLLECT_LTO_WRAPPER=/usr/libexec/gcc/x86_64-redhat-linux/14/lto-wrapper
OFFLOAD_TARGET_NAMES=nvptx-none:amdgcn-amdhsa
OFFLOAD_TARGET_DEFAULT=1
Target: x86_64-redhat-linux
Configured with: ../configure --enable-bootstrap --enable-languages=c,c++,fortran,objc,obj-c++,ada,go,d,m2
,lto --prefix=/usr --mandir=/usr/share/man --infodir=/usr/share/info --with-bugurl=http://bugzilla.redhat.
com/bugzilla --enable-shared --enable-threads=posix --enable-checking=release --enable-multilib --with-sys
tem-zlib --enable-__cxa_atexit --disable-libunwind-exceptions --enable-gnu-unique-object --enable-linker-b
uild-id --with-gcc-major-version-only --enable-libstdcxx-backtrace --with-libstdcxx-zoneinfo=/usr/share/zo
neinfo --with-linker-hash-style=gnu --enable-plugin --enable-initfini-array --with-isl=/builddir/build/BUI
LD/gcc-14.2.1-build/gcc-14.2.1-20250110/obj-x86_64-redhat-linux/isl-install --enable-offload-targets=nvptx
-none,amdgcn-amdhsa --enable-offload-defaulted --without-cuda-driver --enable-gnu-indirect-function --enab
le-cet --with-tune=generic --with-arch_32=i686 --build=x86_64-redhat-linux --with-build-config=bootstrap-l
to --enable-link-serialization=1
Thread model: posix
Supported LTO compression algorithms: zlib zstd
gcc version 14.2.1 20250110 (Red Hat 14.2.1-7) (GCC)
[cozzini@login01 scratch] >
```

Compiling with gcc..

GCC compiler



GCC is a collection that invokes compiler, assembler and linker…

Agenda

- A first look of the software stack ✓
- Local resource manager: queue system ✓
- Scientific software ✓
- Compilers ✓
- Libraries

Scientific Libraries

- Plenty of them for many different tasks
- Dedicated lecture later during the course
- Today let us just focus on static vs dynamic libraries on basic system libraries

Check out: https://medium.com/@abhishekjainindore24/static-library-vs-dynamic-library-understanding-the-differences-26e47cac93b6

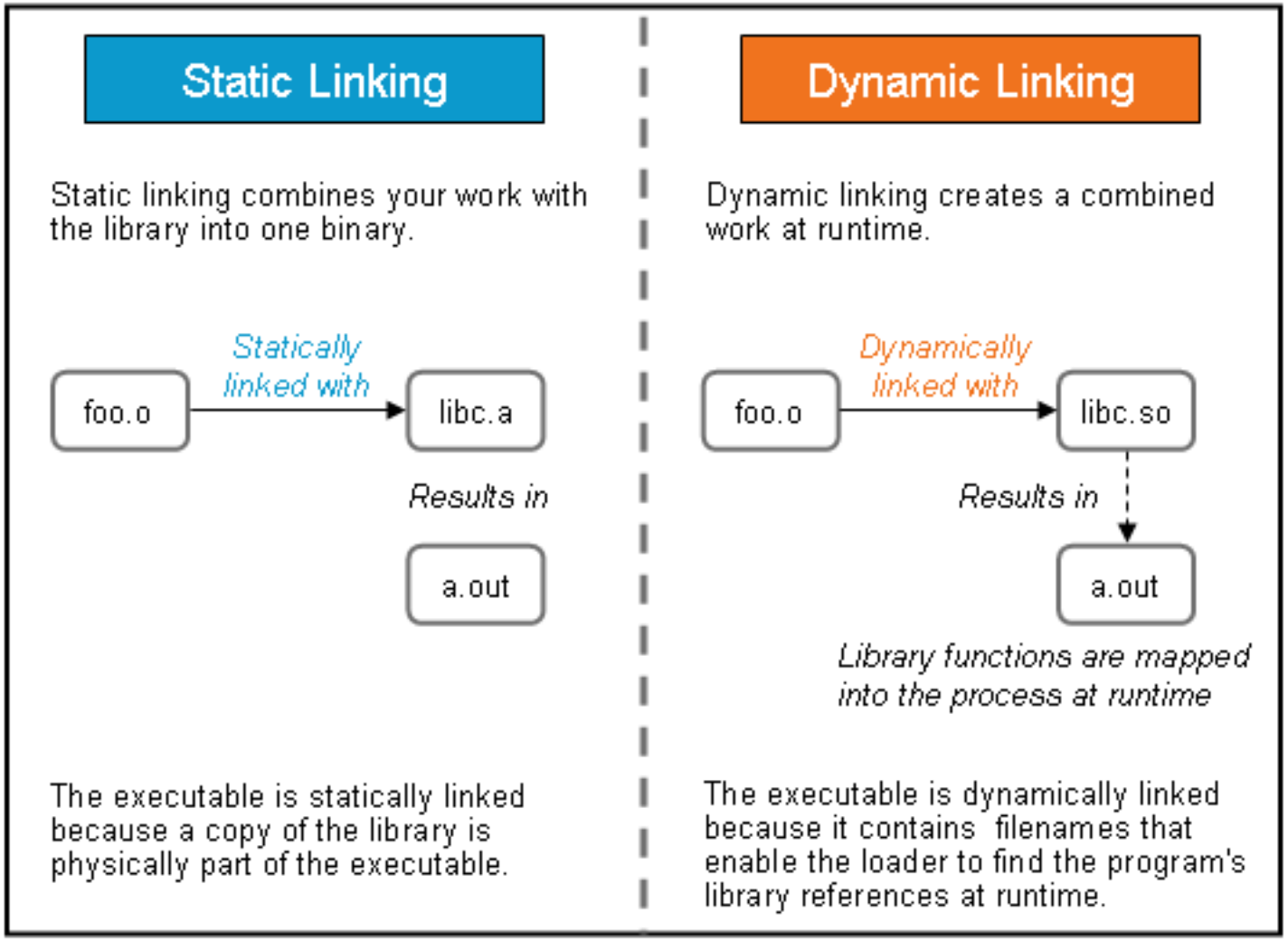
Static libraries: libfoo.o

- .a files are archives of .o files (object files)
- Linker includes needed parts of a static library in the output executable
- No need to find dependencies at runtime – only at build time.
- Can lead to large executables
- Often hard to build a completely static executable on modern systems.

Shared libraries: libfoo.so (Linux)

- More complex build semantics, typically handled by the build system
- Must be found by ld.so and loaded at runtime
- 2 main ways:
  - LD_LIBRARY_PATH: environment variable configured by user and/or module system
  - RPATH: paths embedded in executables and libraries, so that they know where to find their own dependencies.

Comparison


Static Linking
Static linking combines your work with the library into one binary.
foo.o
Statically linked with
libc.a
Results in
a.out
The executable is statically linked because a copy of the library is physically part of the executable.
Dynamic Linking
Dynamic linking creates a combined work at runtime.
foo.o
Dynamically linked with
libc.so
Results in
a.out
Library functions are mapped into the process at runtime
The executable is dynamically linked because it contains filenames that enable the loader to find the program's library references at runtime.

important linux commands

```
> file /usr/bin/cd
/usr/bin/cd: a /usr/bin/sh script, ASCII text executable
> file /usr/bin/pwd
/usr/bin/pwd: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically
linked, interpreter /lib64/ld-linux-x86-64.so.2,
BuildID[sha1]=accef5fc38a136d0affe4dc5638f5f78593fb951, for GNU/Linux 3.2.0,
stripped
> ldd /usr/bin/pwd
linux-vdso.so.1 (0x00007ff8b172d000)
libc.so.6 => /lib64/libc.so.6 (0x00007ff8b1519000)
/lib64/ld-linux-x86-64.so.2 (0x00007ff8b172f000)
```

important linux commands

```
> file xhpl
xhpl: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked,
interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 2.6.32,
BuildID[sha1]=6674352229c114ba61c8f51adb007835911d6d7b, not stripped
>ldd xhpl
linux-vdso.so.1 (0x00007fd469619000)
libpthread.so.0 => /lib64/libpthread.so.0 (0x00007fd4695fe000)
libdl.so.2 => /lib64/libdl.so.2 (0x00007fd4695f9000)
libmpifort.so.12 => not found
libmpi.so.12 => not found
librt.so.1 => /lib64/librt.so.1 (0x00007fd4695f4000)
libm.so.6 => /lib64/libm.so.6 (0x00007fd46950c000)
libiomp5.so => not found
libgcc_s.so.1 => /lib64/libgcc_s.so.1 (0x00007fd4694dd000)
libc.so.6 => /lib64/libc.so.6 (0x00007fd4692ea000)
/lib64/ld-linux-x86-64.so.2 (0x00007fd46961b000)
> ./xhpl
./xhpl: error while loading shared libraries: libmpifort.so.12: cannot open shared object
file: No such file or directory
```

All done !

- A first look of the software stack ✓
- Local resource manager: queue system ✓
- Scientific software ✓
- Compilers ✓
- Libraries ✓